CS293 Project : Plagiarism Checker

Harshil Solanki 23B1016
Dhruvraj Merchant 23B1041
Kunj Bhesaniya 23B0995

November 24, 2024

1 Phase One: Barebones Checking of Two Submissions

1.1 Net Length of Accurate Matches:

Accurate Matches are identified using Rolling Hash Algorithm. Function Signatures used are:

- hashing : Computes the hash value of the first window of a given sequence using modular arithmetic
- calculate_hashes : Generates hash values for all windows of size *len* in the input sequence
- rolling_hash : Finds short matching patterns of varying lengths between two sequences
- upper_nonoverlap_match : Finds the first non-overlapping match after a given match using binary search
- remove_overlap : Removes overlapping matches and maximizes the total length of matches using DP

1.2 Longest Approximate Match and Start Position:

Approximate Matches are identified using Dynamic Programming (DP) and calculating the Longest Common Subsequence (LCS) between two submissions. Function Signatures used are:

- similarity_score : Calculates the similarity score = $\frac{matching\_length}{window\_size}$
- Large_pattern : Finds the largest pattern with a similarity score greater than a threshold
- filter_lcs : Filters the LCS to keep only continuous sequences of at least 10 elements
- longestPattern : Finds the longest pattern match between two submissions using Dynamic Programming (DP)
- evaluate_plag : Evaluates plagiarism based on a threshold: if the short match is at least 60% of the total length, or the large match is at least 50%, it returns true, indicating plagiarism

1.3 Flow:

match_submissions calls rolling_hash which calculates total length of exact matches by calling calculate_hashes and remove_overlap. Matches are stored in a struct which stores the start and end indices in both files. It then calls longestPattern which calculates the LCS and finds the longest approximate match by calling to filter_lcs and Large_pattern. Finally, evaluate_plag is called to use the similarity score based length of matches and print the results.
Overall Time Complexity: $O(nm)$, where n and m are the lengths of the two submissions being compared.
Overall Space Complexity: $O(nm)$, due to the space used by the LCS computation and rolling hash storage.

2 Phase Two: A Full-Fledged Plagiarism Checker

2.1 Long Pattern Matches

The algorithm used to detect long pattern matches between files is based on the rolling hash technique. The function signature is: check_plagiarism. This function calculates the hashes of all substrings of length 15 using the rolling hash technique and compares them between two submissions.

2.2 Patchwork Plagiarism

Patchwork plagiarism is detected by checking if there are more than 20 matches with different submissions. The function signature is: patchWork. This function sorts the intervals of matches and checks for overlapping matches to determine if patchwork plagiarism has occurred.

2.3 Short Pattern Matches

Short pattern matches are handled within the check_plagiarism function. The function signature for handling small matches is handle_small_match. This function attempts to merge small matches with continuous matches.

2.4 Complexity Analysis

2.4.1 Time Complexity

Long Pattern Matches: $O(n \log n)$ for sorting intervals and $O(n)$ for rolling hash calculation per file pair. **Patchwork Plagiarism:** $O(n \log n)$ for sorting intervals. **Short Pattern Matches:** $O(n)$ for each small match check.

2.4.2 Space Complexity

Long Pattern Matches: $O(n)$ for storing hashes and intervals. **Patchwork Plagiarism:** $O(n)$ for storing intervals. **Short Pattern Matches:** $O(n)$ for storing matched indices.

2.5 Threading and Concurrency

Threading and concurrency are used to make `add_submission` calls non-blocking. The `ThreadPool` class manages a pool of threads to handle tasks concurrently. The function signature for enqueuing tasks is:

```
template <class F>
void enqueue(F&& f);
```

This function adds tasks to a queue, which are then processed by the threads in the pool.

2.6 Identifying Files for Plagiarism Check

The set of files to check against is identified based on the submission timestamp. Submissions within one second of the current submission's timestamp are considered for plagiarism checking. This is handled in the `process_plagcheck_for_submissi` function.

2.7 Helpers, Data Structures, and Flow

2.7.1 Helpers and Data Structures

- `Match` struct: Stores match details.
- `ThreadPool` class: Manages a pool of threads.
- `tokenized_submissions`: Stores tokenized submissions.
- `is_flagged`: Tracks flagged submissions.

2.7.2 Flow of the Algorithm

- **Constructor:** Initializes the thread pool and tokenizes initial submissions.
- **add_submission:** Adds a new submission and enqueues a plagiarism check task.
- **process_plagcheck_for_submission:** Processes plagiarism check for a submission.
- **check_plagiarism:** Checks for long pattern matches between two submissions.
- **patchWork:** Checks for patchwork plagiarism.
- **Destructor:** Joins all threads in the thread pool.